

CS224N | Neural Networks and Backpropagation

Selene

August 10, 2026

1 Neural Networks and Backpropagation

1.1 Neural Networks

1.1.1 A Single Neuron

A neuron receives n inputs and produces one output. Different neurons produce different outputs because they have different parameters. A common example is the sigmoid logistic unit. It receives an n -dimensional input vector x and produces an activation value a . The neuron also has an n -dimensional weight vector w and a scalar bias b . Its output is

$$a = \frac{1}{1 + \exp(-(w^\top x + b))}.$$

1.1.2 A Single Layer of Neurons

Now extend this idea to multiple neurons. Suppose their weight vectors are $\{w^{(1)}, w^{(2)}, \dots, w^{(m)}\}$, their biases are $\{b_1, \dots, b_m\}$, and their corresponding activations are $\{a_1, \dots, a_m\}$:

$$\begin{aligned} a_1 &= \frac{1}{1 + \exp(-(w^{(1)\top} x + b_1))}, \\ &\vdots \\ a_m &= \frac{1}{1 + \exp(-(w^{(m)\top} x + b_m))}. \end{aligned}$$

To simplify the notation and make more complex networks easier to describe, define

$$\sigma(z) = \begin{bmatrix} \frac{1}{1 + \exp(-z_1)} \\ \vdots \\ \frac{1}{1 + \exp(-z_m)} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix} \in \mathbb{R}^m,$$

and

$$W = \begin{bmatrix} w^{(1)\top} \\ \vdots \\ w^{(m)\top} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

The pre-activation vector is therefore

$$z = Wx + b,$$

and the vector of sigmoid activations can be written as

$$a = \begin{bmatrix} a_1 \\ \vdots \\ a_m \end{bmatrix} = \sigma(z) = \sigma(Wx + b).$$

1.1.3 Feedforward Computation

The feedforward computation of a neural network is

$$a = f(Wx + b),$$

where x is the input, W and b are parameters, f is an activation function, and a is the hidden-layer activation. The output score is then computed from the hidden layer:

$$s = U^T a = U^T f(Wx + b).$$

The hidden layer applies a nonlinear transformation to extract combinations of input features, allowing the model to learn more complex relationships.

1.1.4 Maximum-Margin Objective

Like most machine-learning models, a neural network needs an optimization objective. A maximum-margin objective encourages the score of a correctly labeled example to exceed that of an incorrectly labeled example. Let s denote the score of the correct example and s_c the score of an incorrect example, where

$$s_c = U^T f(Wx_c + b), \quad s = U^T f(Wx + b).$$

The objective should maximize $s - s_c$, or equivalently minimize $s_c - s$. If we only considered the case $s_c > s$, the objective would be minimize $J = \max(s_c - s, 0)$. However, we want the correct example to score at least a positive margin Δ above the incorrect example. The objective therefore becomes

$$\text{minimize } J = \max(\Delta + s_c - s, 0).$$

1.1.5 Backpropagation

This section discusses how to train the model parameters when the cost J is positive. If the cost is zero, no parameter update is needed. We usually update parameters with gradient descent, so we need the gradient of the loss with respect to every parameter:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta^{(t)}} J.$$

The chain rule allows us to calculate the loss gradient for any parameter used in the feedforward computation.

Notation

Symbol	Meaning
x_i	The i -th input to the neural network
s	The output of the neural network
$z_j^{(k)}$	The scalar input received by neuron j in layer k
$a_j^{(k)}$	The activation output produced by neuron j in layer k
$\delta_j^{(k)}$	The error computed at $z_j^{(k)}$ and propagated backward
$W^{(k)}$	The weight matrix mapping the outputs of layer k to the inputs of layer $k + 1$

The first layer is the input layer, so

$$x_j = z_j^{(1)} = a_j^{(1)}.$$

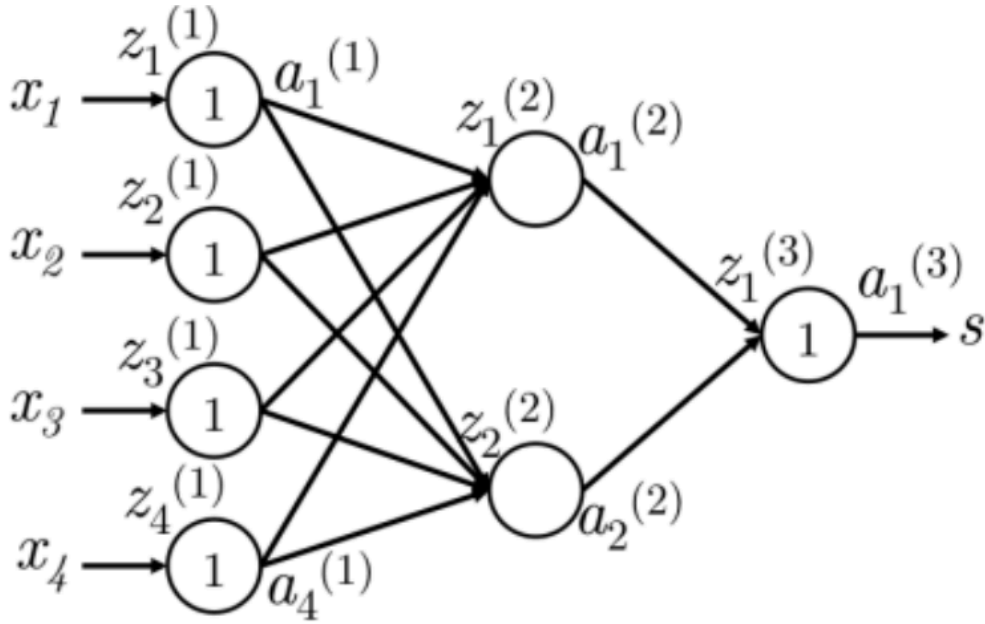


Figure 1: A three-layer neural network and its feedforward connections

Using the notation introduced earlier,

$$W^{(1)} = W, \quad W^{(2)} = U.$$

We now begin the derivation. Assume that $J = 1 + s_c - s > 0$, and consider updating the parameter $W_{14}^{(1)}$. This parameter affects only the input $z_1^{(2)}$ of the first hidden-layer neuron, which in turn affects $a_1^{(2)}$. The gradient therefore propagates backward only along paths influenced by this parameter.

From the maximum-margin loss,

$$\frac{\partial J}{\partial s} = -1, \quad \frac{\partial J}{\partial s_c} = 1.$$

Consider the effect of the first-layer weight $W_{ij}^{(1)}$ on the output s . By the chain rule,

$$\begin{aligned} \frac{\partial s}{\partial W_{ij}^{(1)}} &= W_i^{(2)} \frac{\partial a_i^{(2)}}{\partial W_{ij}^{(1)}} \\ &= W_i^{(2)} \frac{\partial a_i^{(2)}}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{\partial W_{ij}^{(1)}} \\ &= W_i^{(2)} f'(z_i^{(2)}) \frac{\partial z_i^{(2)}}{\partial W_{ij}^{(1)}}. \end{aligned}$$

The input to neuron i in the second layer is

$$z_i^{(2)} = b_i^{(1)} + \sum_k a_k^{(1)} W_{ik}^{(1)},$$

so

$$\frac{\partial z_i^{(2)}}{\partial W_{ij}^{(1)}} = a_j^{(1)}.$$

Substitution gives

$$\frac{\partial s}{\partial W_{ij}^{(1)}} = W_i^{(2)} f'(z_i^{(2)}) a_j^{(1)}.$$

Define the backpropagated error of neuron i in the second layer as

$$\delta_i^{(2)} = W_i^{(2)} f' \left(z_i^{(2)} \right).$$

Then

$$\frac{\partial s}{\partial W_{ij}^{(1)}} = \delta_i^{(2)} a_j^{(1)}.$$

Here, $\delta_i^{(2)}$ is the backward error at neuron i in layer 2, while $a_j^{(1)}$ is the activation from the previous layer that enters the weight $W_{ij}^{(1)}$. In short:

$$\text{weight gradient} = \text{backward error} \times \text{forward input}.$$

For example, $W_{14}^{(1)}$ affects only the first hidden-layer neuron, so the error propagates backward along the corresponding path:

1. Start at the output $a_1^{(3)}$ and propagate an error signal of magnitude 1 backward.
2. As the error passes through $z_1^{(3)} \rightarrow a_1^{(3)}$, multiply it by the local gradient. The local gradient here is 1, so

$$\delta_1^{(3)} = 1.$$

3. The error continues to the previous layer. It is distributed to $a_1^{(2)}$ through the weight $W_1^{(2)}$, giving $W_1^{(2)} \delta_1^{(3)} = W_1^{(2)}$.
4. As the error passes through $z_1^{(2)} \rightarrow a_1^{(2)}$, multiply it by the local derivative $f' \left(z_1^{(2)} \right)$:

$$\delta_1^{(2)} = f' \left(z_1^{(2)} \right) W_1^{(2)}.$$

5. Finally, $W_{14}^{(1)}$ receives $a_4^{(1)}$ during the forward pass, so its gradient is

$$\frac{\partial s}{\partial W_{14}^{(1)}} = \delta_1^{(2)} a_4^{(1)} = a_4^{(1)} f' \left(z_1^{(2)} \right) W_1^{(2)}.$$

Bias Updates

A bias can be viewed as a special weight whose input is always 1. Therefore, the bias gradient of neuron i in layer k is its backpropagated error:

$$\frac{\partial J}{\partial b_i^{(k)}} = \delta_i^{(k)}.$$

For example, the gradient of $b_1^{(1)}$ is

$$\frac{\partial J}{\partial b_1^{(1)}} = \delta_1^{(2)} = f' \left(z_1^{(2)} \right) W_1^{(2)}.$$

General Procedure for Propagating $\delta^{(k)}$ to $\delta^{(k-1)}$

1. The error $\delta_i^{(k)}$ at neuron i in layer k propagates toward the preceding layer along the weight $W_{ij}^{(k-1)}$.
2. Along one connection to neuron j in layer $k-1$, the path contributes the error $\delta_i^{(k)} W_{ij}^{(k-1)}$.
3. Because neuron j in layer $k-1$ is usually connected to several neurons in the next layer, sum the errors from all paths:

$$\sum_i \delta_i^{(k)} W_{ij}^{(k-1)}.$$

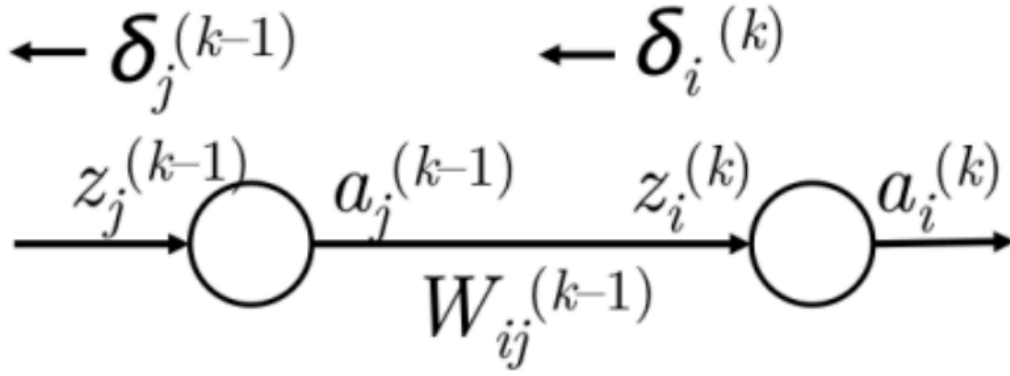


Figure 2: An error signal propagating backward through a weighted connection

4. After obtaining the total error, multiply it by the local derivative $f'(z_j^{(k-1)})$ of that neuron's activation function.
5. The error at neuron j in layer $k - 1$ is therefore

$$\delta_j^{(k-1)} = f'(z_j^{(k-1)}) \sum_i \delta_i^{(k)} W_{ij}^{(k-1)}.$$

1.1.6 Training with Backpropagation: Vectorized Form

So far, we have discussed how to compute the gradient of one parameter. We now generalize the method so that an entire weight matrix can be updated at once.

For a parameter $W_{ij}^{(k)}$, the gradient derived above is $\delta_i^{(k+1)} a_j^{(k)}$. Therefore, the gradient of the entire matrix $W^{(k)}$ is

$$\nabla W^{(k)} = \begin{bmatrix} \delta_1^{(k+1)} a_1^{(k)} & \delta_1^{(k+1)} a_2^{(k)} & \dots \\ \delta_2^{(k+1)} a_1^{(k)} & \delta_2^{(k+1)} a_2^{(k)} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix} = \delta^{(k+1)} a^{(k)\top}.$$

Thus, the matrix gradient is the outer product of the error vector entering the matrix during backpropagation and the activation vector that passed forward through the matrix. The vectorized error recurrence is

$$\delta^{(k)} = f'(z^{(k)}) \circ (W^{(k)\top} \delta^{(k+1)}),$$

where $\circ$ denotes element-wise multiplication.

1.2 Practical Techniques for Neural Networks

1.2.1 Regularization

Like many machine-learning models, neural networks can easily overfit. One common remedy is to add an L_2 regularization penalty to the loss J :

$$J_R = J + \lambda \sum_{i=1}^L \|W^{(i)}\|_F^2.$$

Here, $\|W^{(i)}\|_F$ is the Frobenius norm of $W^{(i)}$, and λ is a hyperparameter controlling the strength of the penalty relative to the original loss. Because the objective minimizes J_R , regularization penalizes excessively large weights while optimizing the loss. The quadratic nature of the penalty reduces the model's flexibility and helps alleviate overfitting.

The Frobenius norm of a matrix U is $\|U\|_F = \sqrt{\sum_i \sum_j U_{ij}^2}$.

This constraint can also be interpreted as a Bayesian prior stating that the optimal weights should be close to zero. How close they should be depends on λ , so choosing an appropriate value requires hyperparameter tuning. If λ is too

large, most weights are forced close to zero and the model cannot learn meaningful information from the training data. It will usually perform poorly on the training, validation, and test sets. If λ is too small, the model may overfit again.

Bias terms are not normally regularized because a bias b controls an overall translation of the model and does not substantially increase its complexity, whereas a weight w determines sensitivity to the input and is more likely to contribute to overfitting.

1.2.2 Dropout

Dropout is a powerful regularization technique with a simple idea: during each forward and backward pass in training, independently discard each neuron with probability $1 - p$, or equivalently retain it with probability p . At test time, use the complete network to compute predictions. Dropout encourages the network to learn more robust representations and can reduce overfitting. It can also be viewed as training an exponential number of smaller subnetworks and approximately averaging their predictions.

In practice, for each layer output h , retain each activation with probability p and otherwise set it to zero. During backpropagation, gradients pass only through neurons retained in the forward pass.

A key detail is that the expected scale of the activations should remain approximately the same during training and testing. The most common implementation is **inverted dropout**, which divides retained activations by p during training and requires no scaling at test time. Alternatively, if activations are not scaled during training, multiply them by p at test time.

1.2.3 Neuron Units

The neural networks discussed so far use sigmoid neurons to introduce nonlinearity. In many applications, however, other activation functions work better. The following functions and their derivatives are common alternatives.

Sigmoid. The sigmoid activation is

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

Its derivative is

$$\sigma'(z) = \frac{\exp(-z)}{(1 + \exp(-z))^2} = \sigma(z)(1 - \sigma(z)).$$

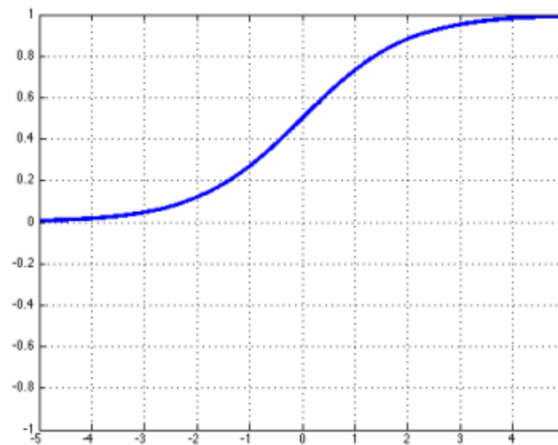


Figure 3: The sigmoid activation function

Tanh. The hyperbolic tangent is an alternative to the sigmoid and often converges faster in practice. Tanh outputs values between -1 and 1 , whereas sigmoid outputs values between 0 and 1 :

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} = 2\sigma(2z) - 1, \quad \tanh(z) \in (-1, 1).$$

Its derivative is

$$\tanh'(z) = 1 - \left(\frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)} \right)^2 = 1 - \tanh^2(z).$$

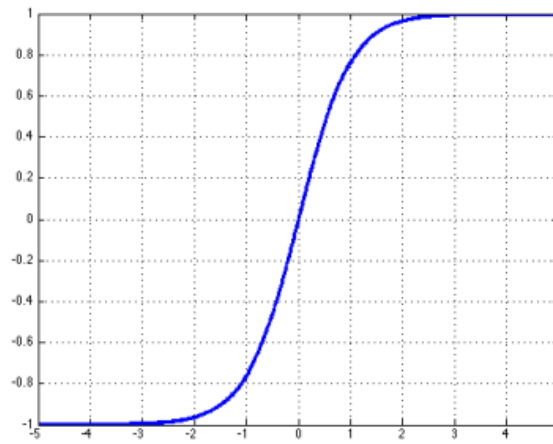


Figure 4: The $\tanh$ activation function

Hard Tanh. Hard Tanh is sometimes used in place of Tanh because it is less expensive to compute, although it saturates when $|z| > 1$:

$$\text{hardtanh}(z) = \begin{cases} -1, & z < -1, \\ z, & -1 \leq z \leq 1, \\ 1, & z > 1. \end{cases}$$

Its derivative is

$$\text{hardtanh}'(z) = \begin{cases} 1, & -1 < z < 1, \\ 0, & \text{otherwise.} \end{cases}$$



Figure 5: The Hard Tanh activation function

Softsign. Softsign is another nonlinear alternative to Tanh. Its tails approach saturation more gradually than those of Tanh:

$$\text{softsign}(z) = \frac{z}{1 + |z|}.$$

Its derivative is

$$\text{softsign}'(z) = \frac{1}{(1 + |z|)^2}.$$

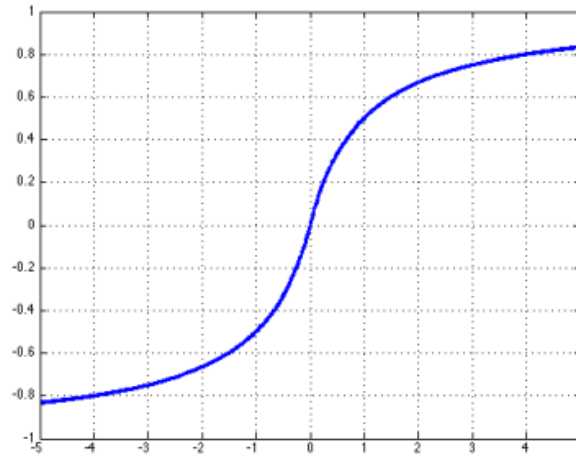


Figure 6: The Softsign activation function

ReLU. The rectified linear unit is a widely used activation function. It does not saturate for large positive values of z and has achieved strong results in computer-vision applications:

$$\text{ReLU}(z) = \max(z, 0).$$

Its derivative is the piecewise function

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

At $z = 0$, a subgradient such as 0 is chosen by convention.

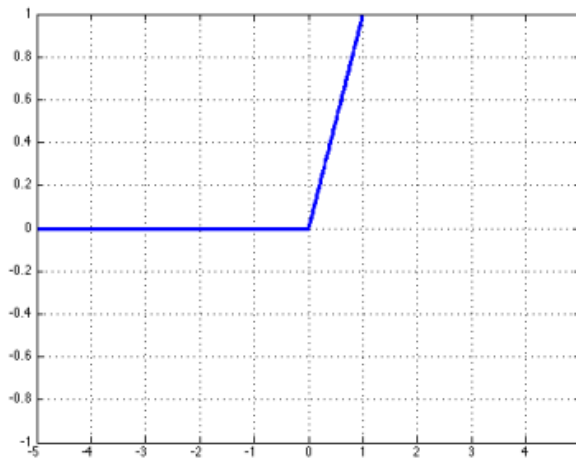


Figure 7: The ReLU activation function

Leaky ReLU. A conventional ReLU does not propagate an error signal when $z < 0$. Leaky ReLU modifies it so that a small gradient can still propagate when z is negative:

$$\text{LeakyReLU}(z) = \max(z, kz), \quad 0 < k < 1.$$

Its derivative is

$$\text{LeakyReLU}'(z) = \begin{cases} 1, & z > 0, \\ k, & z < 0. \end{cases}$$

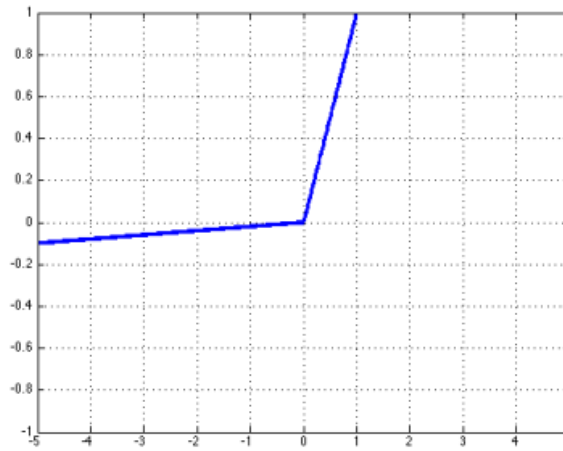


Figure 8: *The Leaky ReLU activation function*

1.2.4 Data Preprocessing

As with other machine-learning models, basic data preprocessing is essential for obtaining reasonable performance on the target task.

Mean subtraction. Given an input dataset X , subtract the mean feature vector from every example so that the data is centered at zero. In practice, compute the mean using only the training set, then subtract that same mean from the training, validation, and test sets.

Normalization. Another common technique, though sometimes used less often than mean subtraction, is to scale every input feature dimension to a similar range. This is useful because input features are often measured in different units, while initially we usually want to treat all features as equally important. Compute each feature's standard deviation on the training set, then divide that feature by the corresponding value across the training, validation, and test sets.

Whitening. Whitening is less common than mean subtraction followed by normalization. It transforms the data so that its covariance matrix is the identity: the features become uncorrelated and each has variance 1. First subtract the mean from the data to obtain X' . Next, compute the singular value decomposition of X' , obtaining U , S , and V . Project X' into the basis defined by the columns of U , then divide each resulting dimension by its corresponding singular value in S . If a singular value is zero, divide by a small positive value instead for numerical stability.